

# Fuzzy Private Set Union avec Chiffrement Homomorphe de Classe $\mathcal{CL}$

Intégration de BICYCL dans volePSI – Comparaison et Évaluation

Yasser NAMATY

Laboratoire Jean Kuntzmann

Encadrants : Aude Maignan / Jean-Guillaume Dumas

2026

# Plan de la présentation

- 1 Contexte et objectifs
- 2 De la Private Set Union à la Fuzzy PSU
- 3 OKVS et volePSI
- 4 Chiffrement homomorphe linéaire : le rôle de BICYCL
- 5 Résultats expérimentaux
- 6 Comparaison BICYCL vs Paillier
- 7 Perspectives : PIR et SimplePIR
- 8 Conclusion

# Contexte et objectifs du projet

## Cadre

- Projet mené au **Laboratoire Jean Kuntzmann (LJK)**
- Encadrement : Aude Maignan, Jean-Guillaume Dumas
- Domaine : cryptographie appliquée, calcul multipartite sécurisé (MPC)

## Objectif

- Intégrer **BICYCL** (chiffrement homomorphe de classe  $\mathcal{CL}$ ) dans un protocole de **Fuzzy Private Set Union** construit sur **volePSI**
- Comparer les performances obtenues à celles du chiffrement de **Paillier**, classiquement utilisé dans ce contexte

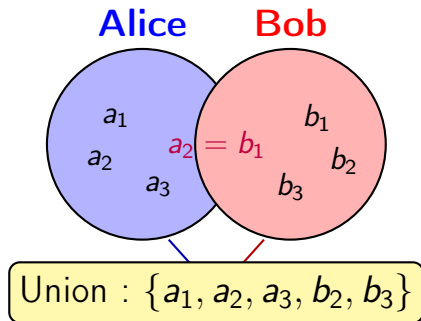
## Pourquoi ce sujet ?

Les protocoles de *Fuzzy* PSU combinent trois briques – structure d'indexation (OKVS), comparaison floue et chiffrement homomorphe – dont le choix conditionne fortement les performances réelles. Comprendre *pourquoi* chaque brique est nécessaire et permet de justifier les choix d'implémentation.

# Private Set Union (PSU) : définition et principe

## Problème

Deux parties, chacune détenant un ensemble d'éléments privés, souhaitent calculer l'union de leurs ensembles sans révéler d'informations supplémentaires.



- Applications : données médicales, détection de menaces, publicité ciblée
- **Objectif** : calculer l'union sans révéler les éléments individuels
- Ici,  $a_2$  et  $b_1$  sont fusionnés car **strictement égaux**

# Limites de la PSU classique : vers la *Fuzzy* PSU

## Pourquoi ce n'est pas suffisant

La PSU classique ne fusionne que des éléments **strictement identiques**. Or beaucoup de données réelles sont **bruitées** : deux mesures du même objet ne sont presque jamais bit-à-bit identiques.

## Exemples de données “proches mais pas égales”

- **Biométrie** : deux scans du même doigt donnent deux empreintes légèrement différentes
- **Géolocalisation** : deux relevés GPS d'un même lieu diffèrent de quelques mètres
- **Génomique** : deux séquences ADN homologues comportent quelques mutations
- **Chaînes de caractères** : fautes de frappe, variantes orthographiques

## Idée de la *Fuzzy* PSU

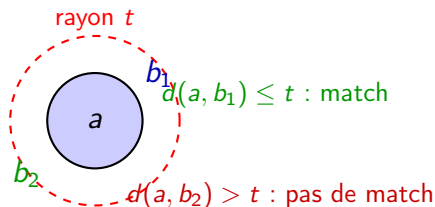
Remplacer l'égalité stricte  $a = b$  par un critère de **proximité** : deux éléments sont fusionnés dès qu'ils sont “assez proches”, selon une distance choisie et un seuil de tolérance.

## Définition

Soit  $(X, d)$  un espace métrique et  $t \geq 0$  un seuil de tolérance. Pour  $x \in A$  et  $y \in B$ , on dit que  $x$  et  $y$  **matchent** si

$$d(x, y) \leq t.$$

La *Fuzzy* PSU calcule une union où les paires  $(x, y)$  qui matchent sont fusionnées, au lieu d'apparaître comme deux éléments distincts. La PSU classique correspond au cas particulier  $t = 0$ .



# Dimension et rayon : deux paramètres déterminants

## La dimension de l'espace $X$

- Chaque élément est représenté par un vecteur de  $X = \mathbb{R}^k$  (ou  $\{0, 1\}^k$ ) :  $k$  est la **dimension** des données (ex.  $k = 2$  pour des coordonnées GPS,  $k$  grand pour un vecteur biométrique)
- **Pourquoi c'est important** : le volume d'une boule de rayon  $t$  croît avec  $k$  – comparer “tout contre tout” devient vite trop coûteux (*malédiction de la dimension*)

## Le rayon (seuil) $t$

- $t$  fixe la tolérance au bruit : trop petit  $\Rightarrow$  on rate des correspondances réelles (faux négatifs) ; trop grand  $\Rightarrow$  on fusionne des éléments qui ne devraient pas l'être (faux positifs)
- **Pourquoi c'est important** :  $t$  détermine directement le nombre de candidats à comparer autour de chaque point, donc le coût du protocole

# Pourquoi une structure clé-valeur pour comparer les ensembles ?

## Le problème du passage à l'échelle

Comparer chaque élément de  $A$  à chaque élément de  $B$  coûte  $\mathcal{O}(n \times m)$ . Pour un protocole utilisable en pratique, il faut une structure qui permette de **retrouver une valeur associée à une clé en temps quasi-constant**, sans comparaison exhaustive.

## Idée

Encoder chaque élément (clé) avec une valeur secrète associée dans une structure de type **dictionnaire clé-valeur**, que l'autre partie pourra interroger de façon **oblivieuse** (sans révéler quelles clés elle possède).

## Vers l'OKVS

Cette structure s'appelle un **Oblivious Key-Value Store (OKVS)** : elle est au cœur de volePSI et du protocole FPSU étudié ici.



# Qu'est-ce qu'un OKVS ?

## Définition

Un **Oblivious Key-Value Store** encode un ensemble de paires  $(k_i, v_i)$  dans une structure  $P$  (un vecteur) telle que :

$$\text{Decode}(P, k_i) = v_i \quad \text{pour toute clé encodée,}$$

et telle que  $P$  ne révèle **rien** sur les clés qui ont servi à le construire.

## Pourquoi “oblivieux” ?

- La structure encodée  $P$  est envoyée en clair (ou combinée à d'autres corrélations) sans que cela ne fuite les clés d'origine
- Seule une partie qui connaît une clé  $k_i$  peut en retrouver la valeur  $v_i$

## Réalisation utilisée : Paxos

volePSI utilise une famille d'OKVS appelée **Paxos**, qui encode  $n$  paires en un vecteur de taille  $\approx 1,03 n$ , avec un encodage et un décodage linéaires en  $n$ .

# Pourquoi un OKVS pour la PSI/PSU ?

## Ce que l'OKVS apporte

- **Compacité** : la structure encodée est à peine plus grande que les données d'origine
- **Rapidité** : encodage et décodage en  $\mathcal{O}(n)$ , contre  $\mathcal{O}(n \times m)$  pour une comparaison exhaustive
- **Confidentialité** : la structure ne révèle pas les clés utilisées pour la construire

## Alternative écartée : le hachage simple

Un simple hachage des éléments permettrait de tester une égalité, mais ne permet ni de transporter une **valeur secrète associée** à chaque clé, ni de combiner cette valeur avec du calcul homomorphe – deux propriétés indispensables au protocole FPSU (voir section suivante).

# volePSI : pourquoi ce protocole ?

## Principe

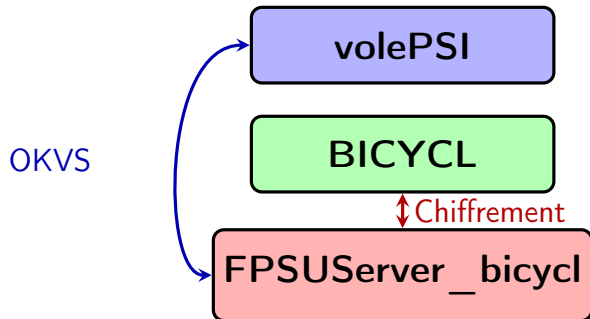
volePSI construit un OKVS à partir de **corrélations VOLE** (Vector Oblivious Linear Evaluation) : chaque partie obtient des parts aléatoires liées par une relation linéaire, sans qu'aucune ne connaisse la part de l'autre.

## Pourquoi ce choix plutôt qu'un OT classique

- Communication et calcul quasi-linéaires en la taille des ensembles
- Sécurité prouvée dans le modèle semi-honnête
- Se combine naturellement avec un OKVS pour transporter des **parts secrètes** ( $s_1, s_2$ ) telles que  $s_1 + s_2 = 0$  en cas de match

## Lien avec le chiffrement homomorphe

Ces parts  $s_1, s_2$  devront être combinées par une **addition** sans être révélées individuellement : c'est le rôle du chiffrement homomorphe additif, présenté plus loin.



- OKVS (Paxos)
- Corrélations VOLE
- $\mathcal{CL}$ -HSMqk

# Pourquoi une somme homomorphe sur les valeurs de l'OKVS ?

## Ce que contient l'OKVS

Chaque serveur encode dans son OKVS des **parts secrètes**  $s_1$  ou  $s_2$  telles que  $s_1 + s_2 = 0$  si et seulement s'il y a un match. Décoder l'OKVS donne accès à ces parts **chiffrées**.

## Pourquoi il faut sommer sans déchiffrer

- Déchiffrer  $s_1$  et  $s_2$  séparément révélerait de l'information sur les entrées de chaque partie
- Il faut donc calculer  $s_1 + s_2$  **directement sur les valeurs chiffrées** extraites de l'OKVS
- Seul le résultat final (match ou non) doit être révélé, jamais les valeurs intermédiaires

## D'où le besoin d'un chiffrement homomorphe additif

L'OKVS résout le problème de l'**indexation** ; le chiffrement homomorphe résout celui de la **combinaison confidentielle** des valeurs qu'il contient. Les deux briques sont complémentaires.

# Le chiffrement homomorphe : principe

## Chiffrement classique

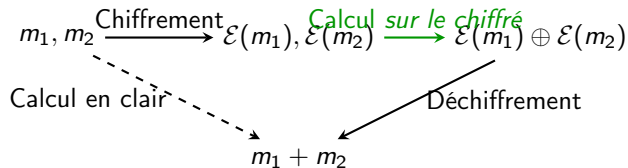
Avec un chiffrement classique (AES, RSA...), le chiffré  $c$  est “opaque” : dès qu’une donnée est chiffrée, on ne peut plus rien calculer dessus sans la déchiffrer d’abord.

## Chiffrement homomorphe

Un chiffrement est **homomorphe** s’il existe une opération  $\oplus$  sur les chiffrés telle que

$$\mathcal{D}(\mathcal{E}(m_1) \oplus \mathcal{E}(m_2)) = m_1 + m_2,$$

sans jamais déchiffrer  $\mathcal{E}(m_1)$  ni  $\mathcal{E}(m_2)$ , et sans connaître la clé privée.



# Rôle du chiffrement homomorphe additif dans FPSU

## Ce qu'il permet

- 1 Chaque serveur chiffre son partage :  $\mathcal{E}(s_1)$ ,  $\mathcal{E}(s_2)$
- 2 Les chiffrés sont **additionnés directement** :  $\mathcal{E}(s_1) \oplus \mathcal{E}(s_2) = \mathcal{E}(s_1 + s_2)$
- 3 Seul le résultat final  $\mathcal{D}(\mathcal{E}(s_1 + s_2))$  est révélé
- 4 Si ce résultat vaut 0  $\Rightarrow$  match ; sinon  $\Rightarrow$  pas de match

## Pourquoi c'est essentiel

Sans homomorphie, il faudrait déchiffrer  $s_1$  et  $s_2$  séparément pour tester l'égalité – ce qui **casserait la confidentialité**. L'homomorphie additive suffit ici : on n'a pas besoin de multiplier des chiffrés, seulement de les sommer.

# Le schéma $\mathcal{CL}$ -HSM<sub>qk</sub> (BICYCL)

## Définition

- Groupe de classes d'idéaux de corps quadratiques imaginaires  $\text{Cl}(\Delta_K)$
- Schéma  $\mathcal{CL}$ -HSM<sub>qk</sub> (Castagnos–Laguillaumie), chiffrement **additif** homomorphe

## Propriété clé

$$\mathcal{E}(m_1 + m_2) = \mathcal{E}(m_1) \oplus \mathcal{E}(m_2)$$

sans multiplication requise, ce qui suffit au test d'égalité de FPSU.

## Caractéristiques (SecLevel 128)

- Chiffrés : 4 663 bits
- Addition homomorphe :  $\approx 11$  ms
- Pas de module secret  $n = pq$  à protéger



# Comment fonctionne BICYCL ? – Chiffrer et déchiffrer

## Deux sous-groupes emboîtés

- Un sous-groupe d'ordre **connu** et premier  $q$  : sert à **encoder le message**  $m$  (via un générateur  $f$  de ce sous-groupe)
- Le reste du groupe, d'ordre **inconnu** : sert à **masquer** le message (aléa de chiffrement)

## Vue d'ensemble du schéma

- 1 **KeyGen** : tire une clé secrète  $x$ , publie  $pk = g^x$  dans le groupe de classes
- 2 **Encrypt**( $m$ ) : tire un aléa  $r$ , calcule  $c_1 = g^r$  et  $c_2 = pk^r \cdot f^m$  (composition de formes, via NUCOMP)
- 3 **Decrypt** : retire le masque avec  $x$ , puis retrouve  $m$  car il vit dans le **petit** sous-groupe d'ordre  $q$  (calcul de logarithme discret facile car  $q$  est connu et petit)
- 4 **Add** : composer directement les formes des deux chiffrés  $\Rightarrow$  chiffré de la somme

# LHE de BICYCL : quel but ?

## Pourquoi ce groupe précis

- L'ordre du groupe de classes est **inconnu** (difficile à calculer), ce qui fonde la sécurité, comme pour RSA
- **Mais sans paramètre secret** : pas de module  $n = pq$  à générer et à protéger (contrairement à Paillier)  $\Rightarrow$  pas de *trusted setup*

## But recherché dans ce projet

- Fournir à FPSU un chiffrement homomorphe additif **plus compact** et **plus rapide à générer/chiffrer** que Paillier
- Éviter la génération d'un module RSA, coûteuse et sensible en sécurité
- Mesurer si ce gain compense un coût d'addition homomorphe plus élevé (voir comparaison)

# BICYCL, share et share chiffré : qui fait quoi ?

## Le *share* (part secrète) : $s_1, s_2$

- Valeur en clair calculée localement par chaque serveur à partir de son ensemble
- Construite de sorte que  $s_1 + s_2 = 0$  si et seulement s'il y a un match
- **Ne doit jamais circuler en clair** entre les deux parties

## Le *share encodé* (share chiffré) : $\mathcal{E}(s_1), \mathcal{E}(s_2)$

- C'est le share, chiffré avec BICYCL ( $\mathcal{CL}$ -HSMqk), puis **sérialisé** au format `ValueType` pour être stocké dans l'OKVS
- C'est cette version chiffrée – pas le share en clair – qui est encodée dans Paxos et transmise

## Rôle de BICYCL dans cette chaîne

- Transforme le share sensible en une donnée manipulable sans le révéler
- Permet de sommer  $\mathcal{E}(s_1)$  et  $\mathcal{E}(s_2)$  directement (homomorphie), puis de ne révéler que le résultat  $\mathcal{D}(\mathcal{E}(s_1) + \mathcal{E}(s_2))$

## Test avec $n = 8$ , $m = 5$

- Indices 0 et 1 : clés synchronisées entre les deux serveurs → **décodage réussi**
- Indice 2 : clés désynchronisées → **échec attendu**
- Indices 3 et 4 : clés aléatoires → **échec attendu**

## Interprétation

- Le décodage de l'OKVS et la reconstruction homomorphe fonctionnent comme prévu
- Les éléments réellement présents dans les deux ensembles sont correctement retrouvés
- Les éléments absents sont correctement rejetés, sans fausse acceptation observée

## Ce qui est comparé

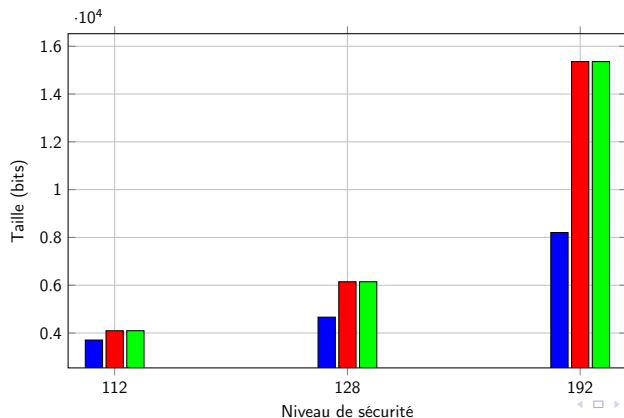
- BICYCL ( $\mathcal{CL}$ -HSMqk) contre deux variantes de Paillier : **PHPE** (Paillier standard) et **SHPE** (Paillier avec précalculs / clé forte)
- Même niveau de sécurité pour une comparaison équitable : **SecLevel 128 bits**

## Métriques mesurées

- Taille des chiffrés (bits) – coût de communication
- Temps de génération de clés, de chiffrement, de déchiffrement (ms)
- Temps d'une addition homomorphe ( $\mu$ s) – opération répétée pour chaque élément de l'OKVS

# Comparaison : taille des chiffrés et génération de clés

|                  | BICYCL | PHPE   | SHPE    | Gain BICYCL |
|------------------|--------|--------|---------|-------------|
| Taille CT (bits) | 4 663  | 6 144  | 6 144   | +24%        |
| KeyGen (ms)      | 24.45  | 183.76 | 1353.04 | ×7-55       |



# Comparaison : temps de chiffrement et déchiffrement

|              | BICYCL | PHPE  | SHPE   | Gain BICYCL      |
|--------------|--------|-------|--------|------------------|
| Encrypt (ms) | 9.44   | 84.75 | 163.63 | ×9–17            |
| Decrypt (ms) | 23.58  | 22.36 | 3.74   | ×6 (défavorable) |

## Lecture

- BICYCL chiffre nettement plus vite que les deux variantes de Paillier
- En déchiffrement, SHPE (avec précalculs) reste plus rapide : c'est le prix de la simplicité de mise en œuvre de BICYCL

# Comparaison : addition homomorphe – le compromis clé

|                 | BICYCL | PHPE / SHPE | Facteur      |
|-----------------|--------|-------------|--------------|
| Add ( $\mu s$ ) | 11 073 | 70          | $\times 158$ |

## Pourquoi cet écart

L'addition dans le groupe de classes de BICYCL repose sur une composition de formes quadratiques (NUCOMP), intrinsèquement plus coûteuse qu'une simple multiplication modulaire chez Paillier.

## Impact sur FPSU

FPSU effectue **une addition par élément de l'OKVS** : ce coût est donc le principal poste de surcoût du protocole avec BICYCL (voir diapositive suivante).



## Synthèse : quel schéma pour quel usage ?

| Critère             | BICYCL         | Paillier    | Vainqueur |
|---------------------|----------------|-------------|-----------|
| Taille des chiffrés | 4 663 bits     | 6 144 bits  | BICYCL    |
| Génération de clés  | 24.45 ms       | 183–1353 ms | BICYCL    |
| Chiffrement         | 9.44 ms        | 85–164 ms   | BICYCL    |
| Déchiffrement       | 23.58 ms       | 3.7–22 ms   | Paillier  |
| Addition homomorphe | 11 073 $\mu$ s | 70 $\mu$ s  | Paillier  |

### Conclusion du benchmark

BICYCL est préférable quand la **compacité** et le **coût de chiffrement** dominant (communication, nombreux clients) ; Paillier reste préférable quand le protocole effectue **beaucoup d'additions homomorphes** par exécution.

# Le Private Information Retrieval (PIR) : principe

## Le problème

Un client souhaite récupérer l'élément d'indice  $i$  dans une base détenue par un serveur, **sans que le serveur apprenne quel indice  $i$  a été demandé.**

## Pourquoi c'est nécessaire ici

Dans FPSU, après le calcul de l'union, un client peut vouloir interroger l'OKVS résultant pour récupérer une entrée précise. Une requête en clair révélerait quel élément l'intéresse – ce qui est exactement ce que le protocole cherche à éviter.

## Intérêt

- Complète les garanties de confidentialité de FPSU jusqu'à la phase de **lecture des résultats**
- Un PIR informationnellement sûr avec des serveurs qui ne collaborent pas garantit qu'aucun d'eux n'apprend l'indice demandé

# SimplePIR : intérêt et perspective d'intégration

## Pourquoi SimplePIR

- Schéma de PIR reposant sur des hypothèses **LWE** standard, réputé pour son excellent compromis simplicité / performance
- Bien adapté à une base de données statique comme l'OKVS produit par FPSU

## État actuel et perspective

- Un premier prototype de SimplePIR a été développé séparément, avec une communication sécurisée entre serveurs et client
- **Prochaine étape** : relier ce module à FPSUServer\_bicycl afin que les requêtes sur l'OKVS passent directement par un canal PIR

## Objectif final

Obtenir une chaîne complète où le client interroge l'union calculée de façon privée de bout en bout : constitution de l'union (FPSU + BICYCL), puis lecture des résultats (SimplePIR).

## Limites actuelles

- Format de valeur transporté par l'OKVS de taille fixe, donc pas optimal en espace
- Tests de validation menés sur de petits ensembles ( $n = 8$ )
- SimplePIR encore non relié à FPSU

## Améliorations possibles

- 1 Compression du format de chiffré transporté
- 2 Chiffrement/déchiffrement par lots (*batch*)
- 3 Passage à l'échelle sur des ensembles de taille réaliste
- 4 Intégration complète du bridge PIR

## Contributions

- ① Intégration de BICYCL dans volePSI pour le protocole FPSU
- ② Campagne de benchmarks BICYCL vs Paillier à niveau de sécurité égal
- ③ Analyse du rôle de chaque brique (OKVS, chiffrement homomorphe, PIR) dans la *Fuzzy* PSU

## Résultats clés

- BICYCL : chiffrés 24% plus compacts, chiffrement jusqu'à  $17\times$  plus rapide
- Coût : addition homomorphe environ  $158\times$  plus lente que Paillier
- **Compromis favorable** pour les applications où la compacité et le chiffrement dominant

Je remercie chaleureusement

**Aude Maignan** et **Jean-Guillaume Dumas**

pour leur encadrement tout au long de ce projet,

ainsi que l'ensemble du **Laboratoire Jean Kuntzmann**  
pour son accueil et les échanges scientifiques.

Merci pour votre attention

**Merci pour votre attention !**

Questions ?